ChatGPT

Related Work

Write Amplification and Compaction Strategies in LSM Stores

LSM-Tree Basics: The Log-Structured Merge tree (LSM) was introduced in the 1990s as a write-optimized index that buffers writes in memory and later merges them to disk in bulk [1]. Google's Bigtable and LevelDB popularized leveled compaction, which keeps one sorted run per level and bounds read amplification at the cost of rewriting data multiple times [2]. In contrast, Apache Cassandra originally adopted size-tiered (tiered) compaction, accumulating multiple runs per level to minimize write work, albeit with higher read costs and occasional large compaction bursts [3]. Over the years, researchers have sought to balance this trade-off by hybridizing strategies. For instance, Dostoevsky (Dayan and Idreos, 2018) proposed making only the largest level leveled and using tiering on all smaller levels, significantly cutting redundant merges and space overhead [3]. This idea was generalized by allowing each level to independently choose a compaction policy (e.g. the LSM-bush approach), forming a continuum between pure leveled and tiered designs [3]. These early efforts laid the groundwork for understanding LSM write amplification (WA) – the excess I/O from re-writing data during compaction – and how compaction policies influence WA and query performance.

Reducing Write Amplification: Several systems introduced new data layouts to attack the root causes of WA. WiscKey (FAST 2016) showed that separating large values from keys can drastically reduce WA by preventing values from being repeatedly rewritten during compaction [4]. By keeping values in a separate log and only sorting keys in the LSM, WiscKey cut write amplification at the cost of an extra read for values, mitigated by sequential value log access and batch prefetching [5] [6]. PebblesDB (SOSP 2017) likewise targets WA with its Fragmented LSM (FLSM) data structure. Instead of fully merging levels, PebblesDB fragments each sorted run into smaller guards that limit the scope of merging [7]. This design "strikes at the root" of WA by **drastically reducing (and often eliminating) data rewrites**, as the LSM no longer merges whole runs but only small chunks guarded by separators [7]. By cutting write I/O in compaction, PebblesDB speeds up merges and increases write throughput, achieving up to 6.7× higher write throughput than RocksDB with 2.4–3× less write I/O [8] [9]. The cost is a moderate loss in worst-case range query efficiency (e.g. ~30% slower for small range scans on a fully compacted store) due to the fragmented layout [10]. Monkey (SIGMOD 2017) approached the WA trade-off analytically: it showed that a leveled LSM can be tuned to an **optimal Pareto point** between update cost and lookup cost by carefully allocating memory across levels [11] [12]. In particular, Monkey assigns progressively tighter Bloom filters to higher (smaller) levels instead of a uniform false-positive rate, shaving an $O(L)$ factor off the worst-case read cost and making lookup cost independent of the number of levels [13] [12]. This optimal filter allocation, combined with tuning of level size ratios and buffer size, lets Monkey minimize read amplification without increasing write amplification. Other notable works include SlimDB (VLDB 2017), which stored **"semi-sorted"** data to save space for cold entries [14], and Lethe (SIGMOD 2020), a delete-aware LSM that treated tombstones differently to avoid excess merging on delete-heavy workloads [15]. Collectively, these foundational efforts shaped our understanding of LSM tuning: they either reduce WA (WiscKey, PebblesDB, Dostoevsky) or optimize memory/compaction parameters (Monkey, SlimDB) to improve the write-read trade-off. However, most assume steady-state or worst-case scenarios, rather than modeling moment-to-moment performance.

Mitigating Write Stalls and I/O Contention

While LSM-trees excel at high throughput, they often suffer **write stalls** – periods where incoming writes pause because background compactions can't keep up [16] [17]. Chen Luo et al. (PVLDB 2019) provided an early deep dive into this issue of performance instability in LSM storage. They observed RocksDB's instantaneous write rate oscillating as compactions periodically caused stalls, and argued that some stalling is inevitable unless the ingest rate stays below the LSM's processing capacity [18] [17]. Their work introduced a simple two-phase method to evaluate stalls for any LSM design: (1) run a burst of writes to identify the maximum sustainable throughput before stalls, then (2) drive the LSM at that rate to measure stall frequency and latency impact [19] [17]. They also explored how different merge scheduler designs (e.g. compaction throttling or parallelism) can minimize stalls under a given I/O bandwidth budget [20]. The key finding was that smarter scheduling of flushes/compactions can smooth out the write rate and significantly reduce tail latency impact without changing the underlying LSM structure [18] [17]. This idea of managing foreground–background interference has been embraced by subsequent systems.

I/O Scheduling and Isolation: SILK (SIGMOD 2020) specifically tackled the high tail latencies caused by compaction interference in RocksDB and Cassandra. It introduced an adaptive I/O scheduler that coordinates client writes with internal operations to **prevent latency spikes** [21] [22]. SILK's scheduler uses three techniques: (i) dynamic bandwidth allocation – it gives compactions more disk share during lulls and throttles them during peak loads; (ii) prioritization of critical internal tasks – it ensures long-blocking operations (like lower-level merges that might fill up L0) run to completion to avoid cascading stalls; and (iii) preemption of less urgent work – it can pause non-critical compactions when user requests surge [22]. Together these strategies reduced write-induced query slowdowns, yielding far more stable tail latencies under mixed workloads [23] [22]. Similarly, bLSM (SIGMOD 2012) had earlier introduced a geared merge scheduler to provide near-constant ingestion by smoothing out bursts, essentially implementing a form of admission control on writes to avoid ever hitting an L0 overflow condition. These approaches treat the LSM like a queuing system, dynamically balancing foreground and background I/O to keep latency low – but they do so empirically, without an explicit performance model.

Multi-Tier and Transient Solutions: Other work addresses stalls by redesigning LSM storage across multiple hardware tiers. NoveLSM (2019) and NVM-Rocks are examples that incorporate NVM (non-volatile memory) as an intermediate store to absorb bursts. NoveLSM keeps a large mutable memtable in byte-addressable NVM, delaying flushes to SSD [24] [25]. This lowers the frequency of flush stalls and initially improves throughput. However, because it flushes much bigger sorted runs to L0, the subsequent L0–L1 compactions become extremely heavy – ironically **exacerbating write stalls** when they do occur [25]. The lesson was that simply adding an NVM buffer can postpone but not eliminate the inevitable stall, and may worsen worst-case pauses. Building on that insight, MatrixKV (USENIX ATC 2020) proposed a more structured NVM utilization to truly smooth out stalls. MatrixKV elevates the entire L0 to NVM and manages it with a specialized **matrix container** data structure [26]. Incoming memtables are flushed into the NVM receiver component; once full, the receiver becomes a read-only compactor that gradually merges its data with the next level on SSD [27]. Crucially, MatrixKV performs fine-grained column compactions – it partitions L0–L1 merges by key ranges so that each compaction job rewrites only a fraction of the L0 data [26] [28]. This reduces the amount of data per compaction and thus shortens stall duration. MatrixKV also "flattens" the LSM by using the NVM tier to effectively reduce the number of levels on SSD, further cutting write amplification [28] [29]. The result is a much more **predictable latency profile**, though at the cost of some extra writes (the matrix container's two-phase flush incurs redundant writes) [28]. Building on MatrixKV, TriangleKV (IEEE TPDS 2022) introduced an even more efficient L0 structure (a triangle container) and compaction method [30]. TriangleKV relocates L0 entirely in NVM like MatrixKV, but its "right-angle" compaction breaks L0–L1 merging into even finer-grained keyrange chunks and uses a widening LSM geometry (fewer levels with higher capacity each) to **mitigate both stalls and overall write amplification** [30] [28]. Thanks to these optimizations, TriangleKV achieved order-of-

magnitude latency improvements over stock RocksDB – e.g. **5.5× lower 99th-percentile write latency than RocksDB on SSD**, and significant gains even over prior NVM-based systems (2× versus NoveLSM, 1.1× versus MatrixKV) [31] [25]. These multi-tier designs demonstrate the benefit of performance isolation: by handling L0 bursts in a faster medium and cleverly scheduling compactions, they nearly eliminate user-visible stalls. However, they require additional hardware (NVM) and complex structural changes. Moreover, their focus is still on reducing or hiding stalls via design – none provide a general model to predict when stalls will happen under a given workload.

Learned and Adaptive LSM Optimization

Another avenue of recent work has been applying machine learning to automate LSM tuning or even to adapt the LSM structure on the fly. Traditional LSM systems expose dozens of tuning knobs (RocksDB has >100) that control memory allocation, compaction scheduling, Bloom filters, etc. [32] [33]. Configuring these for a given workload is notoriously hard. Recent research has therefore explored both black-box and white-box ML techniques to optimize LSM performance without manual intervention. **Workload tuning** tools like **RTune** and **K2V-Tune** (2021–2022) use techniques such as Bayesian optimization or deep learning to recommend RocksDB knob settings that maximize throughput for a target workload, but these typically handle a limited subset of parameters (e.g. cache sizes, Bloom filters) [34] [35]. In contrast, Thakkar et al. (HotStorage 2024) propose using Large Language Models (LLMs) to tackle the full configuration space. By leveraging an LLM "trained" on documentation and source code of LSM systems, their approach (LLM-Tune) generates tuned configs from high-level workload descriptions [36] [37]. Initial results show substantial benefits – up to **3× higher throughput and 9× lower p99 latency** compared to RocksDB's default autotuner on various workloads [37]. This LLM-based method is intriguing as it injects external knowledge into the tuning process, but it remains a heuristic approach (relying on the LLM's internal model of system behavior rather than a formal performance model).

Perhaps the most ambitious is **RusKey** (SIGMOD 2024), which embeds learning inside the LSM engine to continually adapt its behavior. RusKey is described as the **first reinforcement learning (RL)-based key-value store** that can reconfigure its LSM tree in real-time to handle dynamic workloads [38]. Unlike prior static designs, RusKey's LSM can fluidly switch between compaction strategies (e.g. between leveling and tiering) as workload patterns change. To enable this, the authors design a new FLSM-tree variant that permits on-the-fly transitions with minimal overhead [38]. An RL agent observes the workload and chooses actions (like "switch LSM to leveled mode" or "use larger memtable") to maximize long-term throughput; the FLSM structure ensures these transformations happen quickly without massive compactions [38]. Notably, RusKey requires no prior workload knowledge or training traces, learning optimal compaction policies online via reward feedback [38]. In experiments it maintained significantly steadier performance under workload shifts, and delivered up to **4× better throughput than RocksDB** on a variety of workload mixes [38]. RusKey's approach is a form of closed-loop control for LSM systems – rather than modeling performance analytically, it learns the optimal tuning through trial-and-error. A limitation, however, is complexity: the system must explore and occasionally make suboptimal decisions while learning, and the convergence time or stability of the RL policy can be hard to guarantee for all workloads. Still, it represents a major step toward self-driving storage engines. Another learned approach is to use off-line training to guide on-line behavior: for example, researchers have trained models to predict LSM query latency or write cost under certain configurations, and then use these models for knob tuning or scheduling decisions (a form of learned cost modeling). While promising, these techniques often struggle with generality – an ML model might only be valid for the range of workloads it saw in training. In summary, ML and RL techniques are beginning to tame LSM tuning complexity and enable adaptive behavior. They complement analytical modeling by offering solutions where formal models are elusive (e.g. highly complex parameter interactions), but at the cost of requiring extensive experimentation and careful validation to trust the learned policies.

LSM Trees in Modern Systems and Hardware

Broader Systems and Engines: The prevalence of LSM-tree storage in industry underscores the need for robust performance models. RocksDB, LevelDB, Cassandra, HBase, and many other systems all use LSM under the hood [2], each adding their own tweaks. For example, RocksDB introduced features like **bulk insert scheduling, compaction pipelining, and tombstone compaction** to mitigate some LSM costs in production, and it offers multiple compaction styles (leveled, FIFO, universal) to suit different scenarios. Cassandra toggles between size-tiered and leveled compaction based on use-case (size-tiered for write-heavy workloads, leveled for read-intensive ones), and its designers have published lessons on tuning compaction throughput to avoid impacting query latencies. MongoDB's WiredTiger engine provides an LSM option but by default sticks with B+trees, partly because LSM's benefits don't always outweigh the read amplification for general workloads – an insight that again points to the need for predictive modeling of when LSM is advantageous. Facebook's MyRocks showed that an LSM engine can be integrated into a SQL database (MySQL) and deliver huge space savings, but required careful parameter tuning (e.g. bloom filters, cache) to achieve performance parity with traditional engines for read-heavy queries. These experiences, spread across disparate systems, highlight that there is no one-size-fits-all LSM configuration – further motivating a model-based approach to predict performance under different configs and workload mixes.

Advances in Storage Hardware: The landscape of storage hardware has also evolved, and researchers have explored how LSM performance changes (and can be improved) on new devices. One trend is **storage disaggregation** – using remote network-attached storage instead of local SSDs. Meta recently reported their experience in building a disaggregated RocksDB service on cloud storage [39]. They found that RocksDB's append-heavy, sequential write patterns align well with network storage, but metadata operations and tail latencies needed optimization. By tweaking RocksDB's compaction logic and employing a specialized distributed filesystem (Tectonic) underneath, they restored much of the lost performance and enabled RocksDB to run efficiently in a remote-storage configuration [39]. Another trend is making LSMs flash-friendlier. Modern NVMe SSDs offer features like **Zoned Namespaces (ZNS)**, which allow append-only writes to specific zones on disk. Lee et al. (2023) leveraged this in WALTZ, an LSM variant for ZNS SSDs [40]. By using the ZNS zone append command for writes, WALTZ lets the device internally manage write ordering and parallelism, eliminating the need for RocksDB's costly write-group commit synchronization [40]. This cut tail write latencies by about **2–3×** and even improved throughput modestly [41]. Similar work on ZNS (e.g. SplitLSM) shows that aligning LSM compaction with the device's zone layout can reduce garbage collection overheads and smooth performance [42] [43]. Researchers have also proposed key-value SSDs that implement part of the LSM logic in hardware – for instance, Samsung's KV-SSD can offload point lookups and eliminate I/O amplification from key-value translation. Recent systems like Dotori (2023) combine an on-drive B+-Tree with host-side LSM to exploit such devices [44]. These approaches aim to push performance (especially read latency) beyond what is possible with a pure software LSM on block SSDs. Finally, academic engines like SplinterDB (OSDI 2023) revisit the LSM design itself in light of fast storage. SplinterDB introduced the concept of **quotient filters (maplets)** to replace Bloom filters and enable a novel compaction policy [45] [46]. In SplinterDB's mapped mode, each level can hold many runs (like tiered compaction) to keep write amplification low, but all runs share a single merged filter (the quotient maplet) that is as effective as if the level were fully compacted [46]. These maplets can be maintained and merged very cheaply, decoupling the cost of compaction from query overhead [46]. Thus, Mapped SplinterDB claims to get the best of both worlds: insertion throughput comparable to a lazy (tiered) scheme and query performance close to an aggressive (leveled) scheme. Empirical results showed it beating RocksDB by up to **9× on write throughput** and ~**83% on read throughput** in certain benchmarks [47]. This underscores that even in 2023, innovations in data structures are still advancing the state of the art for LSM performance. However, such systems are complex and their benefits can be workload-dependent; a robust performance model could help decide when a Splinter-like approach is worthwhile.

Summary and Distinction: Prior work spans a wide spectrum – from fundamental LSM tuning and compaction strategies, to scheduling techniques for stability, to machine-learned adaptivity, to leveraging new hardware capabilities. Each thread of research improves some aspect of LSM performance, but typically in a specific context or by adding new mechanisms. In contrast, **our work (the RocksDB Put-Rate model)** takes a complementary approach: rather than introduce a new compaction algorithm or tuner, we develop an analytical model of real-time LSM performance. This model captures the transient dynamics of RocksDB's write path – how memtable flushes, compaction backlogs, and I/O contention with reads interplay to determine the instantaneous put throughput. By providing a predictive framework (grounded in first principles and empirical validation), our work enables understanding when and why stalls or performance drops happen under mixed workloads, and how close the system is to its saturation point. Such insight is difficult to obtain from prior solutions: empirical tuners and RL systems treat the LSM as a black box, while structural innovations often sidestep worst-case analysis. In summary, the **RocksDB Put-Rate Model** differentiates itself by offering a unifying performance model that applies to unmodified production LSM engines, allowing practitioners to anticipate performance issues and guide dynamic optimizations in a principled way, whereas previous approaches either improved performance in specific scenarios or relied on reactive adjustments without a comprehensive predictive theory. (Our model can, for example, explain the onset of compaction stalls observed in studies like Luo's [16], and guide how to adjust the arrival rate or compaction throughput to avoid them – something prior art addresses only indirectly.) Thus, we believe our work fills a crucial gap by tying together the transient behaviors and mixed I/O interference in LSM systems under one analytical umbrella, complementing the wealth of existing techniques in this space.

Sources:

- O'Neil et al. "The Log-Structured Merge-Tree (LSM-tree)." Acta Informatica, 1996.
- Chang et al. "Bigtable: A Distributed Storage System for Structured Data." OSDI 2006.
- Sears and Ramakrishnan. "bLSM: A General Purpose Log Structured Merge Tree." SIGMOD 2012.
- Dayan and Idreos. "Dostoevsky: Better Space-Time Trade-offs via Adaptive Merging." SIGMOD 2018 [48].
- Srinivasan et al. "PebblesDB: Building Key-Value Stores using Fragmented LSM-Trees." SOSP 2017 [7] [8].
- Lu et al. "WiscKey: Separating Keys from Values in SSD-Conscious Storage." FAST 2016 [4].
- Yang et al. "SlimDB: A Space-Efficient Key-Value Storage Engine for Semi-Sorted Data." PVLDB 2017 [14].
- Sarkar et al. "Lethe: A Tunable Delete-Aware LSM Engine." SIGMOD 2020 [15].
- Liu et al. "LSM-tree Based Storage Techniques: A Survey." arXiv 2022.
- Luo and Carey. "On Performance Stability in LSM-based Storage Systems." PVLDB 2019 [18] [17].
- Balmau et al. "SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores." SIGMOD 2019/2020 [21] [22].
- Yao et al. "MatrixKV: Reducing Write Stalls and Write Amplification in LSM-tree KV Stores." USENIX ATC 2020 [26] [28].
- Ding et al. "TriangleKV: Reducing Write Stalls and WA in LSM KVs with NVM." IEEE TPDS 2022 [30] [31].
- Mo et al. "RusKey: Reinforcement Learning to Optimize LSM-trees under Dynamic Workloads." SIGMOD 2024 [38].
- Thakkar et al. "Can Modern LLMs Tune and Configure LSM-based KV Stores?" HotStorage 2024 [32] [37].
- Dong et al. "Disaggregating RocksDB: A Production Experience." CIDR 2023 (Meta) [39].
- Lee et al. "WALTZ: Leveraging Zone Append to Tighten Tail Latency of LSM on ZNS SSDs." HotStorage 2023 [40] [41].

- Conway et al. “SplinterDB and Maplets: Improving the Trade-offs in KV Store Compaction Policy.” ATC 2023 [46] [47] .
- **RocksDB Put-Rate Model paper (this work).**

---

[1] [16] [17] [18] [19] [20] vldb.org
https://www.vldb.org/pvldb/vol13/p449-luo.pdf

[2] [3] [14] [15] [48] vldb.org
https://vldb.org/pvldb/vol14/p2216-sarkar.pdf

[4] [5] [6] usenix.org
https://www.usenix.org/system/files/conference/fast16/fast16-papers-lu.pdf

[7] [8] [9] [10] PebblesDB: Building Key-Value Stores using Fragmented Log-Structured Merge Trees
https://www.cs.utexas.edu/~vijay/papers/sosp17-pebblesdb.pdf

[11] [12] [13] courses.cs.washington.edu
https://courses.cs.washington.edu/courses/csep544/21sp/papers/monkey-stratos-idreos-sigmod-2017.pdf

[21] [22] [23] LSM-Tree Key Value Store Literature Review
https://kelk.ai/assets/files/Literature_Review_Ian_Kelk-113a3e429c682fca8503765411a7bcdd.pdf

[24] [25] [26] [27] [28] [29] [30] [31] TriangleKV: Reducing Write Stalls and Write Amplification in LSM-Tree Based KV Stores With Triangle Container in NVM
https://ranger.uta.edu/~jiang/publication/Journals/2022/IEEE-TPDS(TriangleKV).pdf

[32] [33] [34] [35] [36] [37] Can Modern LLMs Tune and Configure LSM-based Key-Value Stores?
https://asu-idi.github.io/publications/files/HS24_GPT_Project.pdf

[38] (PDF) Learning to Optimize LSM-trees: Towards A Reinforcement Learning based Key-Value Store for Dynamic Workloads
https://www.researchgate.net/publication/373116741_Learning_to_Optimize_LSM-trees_Towards_A_Reinforcement_Learning_based_Key-Value_Store_for_Dynamic_Workloads

[39] [40] [41] [42] [43] [44] [45] [46] [47] Rethinking LSM-tree based Key-Value Stores: A Survey
https://www.researchgate.net/publication/393684380_Rethinking_LSM-tree_based_Key-Value_Stores_A_Survey